



**Tecnológico
de Monterrey**

**Programación de estructuras de datos y
algoritmos fundamentales**

Gpo 850

Después de clase | Act-Integradora-3 Estructuras
de datos jerárquicas

Reflexión

Alumno: Carlos Alberto Pérez Díaz

Matrícula: A01204060

Fecha: 15/05/2026

A. Importancia y eficiencia del uso de estructuras jerárquicas

Las estructuras de datos jerárquicas son fundamentales en problemas donde se necesita organizar información de manera eficiente y priorizada. En esta actividad, el uso de estructuras jerárquicas permitió manipular grandes cantidades de registros de una bitácora de accesos IP de forma más eficiente que utilizando estructuras lineales tradicionales.

El Binary Heap resultó especialmente útil debido a que permite acceder rápidamente al elemento con mayor prioridad, en este caso las direcciones IP con mayor número de accesos. Gracias a la organización jerárquica del Heap, operaciones como insertar elementos o eliminar el máximo elemento pueden realizarse en tiempo logarítmico, lo cual mejora considerablemente el desempeño del programa al trabajar con miles de registros.

Además, el algoritmo Heap Sort permitió ordenar los registros de la bitácora por dirección IP de manera eficiente, manteniendo una complejidad temporal de $O(n \log n)$, lo cual es adecuado para conjuntos grandes de información como los utilizados en esta actividad.

B. ¿Por qué usar un Binary Heap y no un BST?

En esta solución fue más conveniente utilizar un Binary Heap en lugar de un Binary Search Tree (BST) debido a la naturaleza del problema. El objetivo principal era obtener de forma eficiente las IPs con mayor número de accesos, por lo que resultaba importante recuperar constantemente el elemento de mayor prioridad.

El Binary Heap permite realizar la operación `getTop()` en tiempo $O(1)$, mientras que insertar y eliminar elementos se realiza en $O(\log n)$. Esto lo convierte en una estructura muy eficiente para implementar colas de prioridad.

Por otro lado, un BST puede presentar problemas de balanceo. En el peor de los casos, un BST no balanceado puede degradarse hasta tener complejidad $O(n)$ en operaciones de búsqueda, inserción o eliminación. Aunque existen variantes balanceadas como AVL o Red-Black Tree, su implementación es más compleja.

En comparación, el Binary Heap ofrece una solución más simple y eficiente para el problema específico de obtener las IPs con mayor prioridad.

C. Complejidad computacional de las operaciones

Las operaciones principales utilizadas en la implementación fueron push, pop y getTop.

push()

La operación push() inserta un nuevo elemento dentro del Heap y posteriormente reorganiza la estructura para conservar la propiedad del MaxHeap.

Complejidad temporal: $O(\log n)$

pop()

La operación pop() elimina el elemento con mayor prioridad, ubicado en la raíz del Heap. Después de eliminarlo, el Heap debe reorganizarse nuevamente.

Complejidad temporal: $O(\log n)$

getTop()

La operación getTop() únicamente accede al elemento raíz del Heap.

Complejidad temporal: $O(1)$

Heap Sort

El algoritmo Heap Sort fue utilizado para ordenar los registros de la bitácora por dirección IP.

Complejidad temporal: $O(n \log n)$

Su impacto

El uso de estas operaciones permitió que la solución tuviera un buen rendimiento incluso con una gran cantidad de registros. Gracias a Heap, fue posible recuperar rápidamente las IPs más frecuentes sin necesidad de ordenar repetidamente los accesos.

D. Pseudocódigo para encontrar la IP con menor número de accesos

Pseudocódigo:

menorAcceso $\leftarrow$ infinito

ipResultado $\leftarrow$ ""

PARA cada elemento en accesos HACER

SI accesos ≥ 3 ENTONCES

SI accesos $<$ menorAcceso ENTONCES

menorAcceso $\leftarrow$ accesos

ipResultado $\leftarrow$ ip

FIN SI

FIN SI

FIN PARA

MOSTRAR ipResultado y menorAcceso

Complejidad temporal:

$O(n)$

Esto se debe a que únicamente se realiza un recorrido lineal sobre el vector de accesos.

Conclusión

En esta actividad fue posible aplicar estructuras de datos jerárquicas para resolver un problema de análisis de registros de red de manera eficiente. Usar Heap Sort permitió ordenar grandes cantidades de información, mientras que el Binary Heap facilitó la recuperación de las IPs con mayor prioridad.

Además de reforzar conceptos de complejidad computacional, esta actividad me permitió comprender cómo estructuras como los Heaps pueden utilizarse en situaciones reales relacionadas con análisis de tráfico y procesamiento de datos.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (3rd Edition).

Weiss, M. A. Data Structures and Algorithm Analysis in C++.

GeeksforGeeks. Binary Heap.

<https://www.geeksforgeeks.org/dsa/binary-heap/>

GeeksforGeeks. Heap Sort.

<https://www.geeksforgeeks.org/dsa/heap-sort/>

cppreference. `std::vector`.

<https://en.cppreference.com/w/cpp/container/vector>

cppreference. `std::fstream`.

https://en.cppreference.com/w/cpp/io/basic_fstream